

can use operations directly to represent the work of the model. For example:

```
// Define function
function predict(input) {
  // y = a * x ^ 2 + b * x + c
  // More on tf.tidy in the next section
  return tf.tidy(() => {
    const x = tf.scalar(input);

    const ax2 = a.mul(x.square());
    const bx = b.mul(x);
    const y = ax2.add(bx).add(c);

    return y;
  });
}

// Define constants: y = 2x^2 + 4x + 8
const a = tf.scalar(2);
const b = tf.scalar(4);
const c = tf.scalar(8);

// Predict output for input of 2
const result = predict(2);
result.print() // Output: 24
```

Or you can use APIs like *tf.model* and *tf.sequential* to construct a model. The following code constructs a *tf.sequential* model:

```
const model = tf.sequential();
model.add(
  tf.layers.simpleRNN({
    units: 20,
    recurrentInitializer: 'GlorotNormal',
    inputShape: [80, 4]
  })
);

const optimizer = tf.train.sgd(LEARNING_RATE);
model.compile({optimizer, loss: 'categoricalCrossentropy'});
model.fit({x: data, y: labels});
```

There are many kinds of layers available in TensorFlow.js. A few examples include *tf.layers.simpleRNN*, *tf.layers.gru* and *tf.layers.lstm*.

Memory management

TensorFlow.js uses the GPU to accelerate math operations, so it's necessary to manage GPU memory while working with tensors and variables.

TensorFlow.js provides two functions for this.

Dispose: You can call *dispose* on a tensor or variable to

purge it and free up its GPU memory:

```
const x = tf.tensor2d([[0.0, 2.0], [4.0, 6.0]]);
const x_squared = x.square();

x.dispose();
x_squared.dispose();
```

Tf.tidy: Using *dispose* can be cumbersome when doing a lot of tensor operations. TensorFlow.js provides another function, *tf.tidy*, which plays a similar role to regular scopes in JavaScript, but for GPU-backed tensors.

tf.tidy executes a function and purges any intermediate tensors created, freeing up their GPU memory. It does not purge the return value of the inner function.

tf.tidy takes a function to tidy up after:

```
const average = tf.tidy(() => {
```

tf.tidy will clean up all the GPU memory used by tensors inside this function, other than the tensor that is returned.

Even in a short sequence of operations like the one below, a number of intermediate tensors get created. So it is a good practice to put your math ops in a *tidy*!

```
const y = tf.tensor1d([1.0, 2.0, 3.0, 4.0]);
const z = tf.ones([4]);

return y.sub(z).square().mean();
});
```

```
average.print() // Output: 3.5
```

Using *tf.tidy* will help prevent memory leaks in your application. It can also be used to more carefully control when memory is reclaimed. The function passed to *tf.tidy* should be synchronous and also not return a *promise*.

tf.tidy will not clean up variables. As variables typically last through the entire life cycle of a machine learning model, TensorFlow.js doesn't clean them up even if they are created in a *tidy*; however, you can call *dispose* on them, manually. **END** 

References

- [1] <https://js.tensorflow.org/>
- [2] <https://opensource.google.com/projects/tensorflow>
- [3] <https://github.com/tensorflow/tensorflow>
- [4] <https://www.expertsystem.com/machine-learning-definition/>

 By: Abhinav Nath Gupta

The author is a software development engineer at Cleo Software India Pvt Ltd, Bengaluru. He is interested in studying machine learning and artificial intelligence techniques.